

RESUME

RESUME — session resumption record (§11.4.131)

Rev 27 · 2026-09-07T10:02:04Z. Supersedes rev 26 (2026-09-07T08:58:39Z). **Rev 27 records one state change and nothing else: THE BATCH IS NOW COMMITTED. It is NOT pushed.** Rev 26 said “THE BATCH IS NOT FINISHED. NOTHING IS COMMITTED. NOTHING IS PUSHED.” — landing the batch falsified that sentence, and rev 26 itself named this sync as the required next action. That correction is rev 27’s whole purpose; no engineering work is claimed by it.

The batch landed as 15 commits on meta main (926eaf3e..dfb06d38), plus f3f0540b for the one determinism guard that arrived after its group had already been committed. Every substantive claim below — the gate/redaction/determinism work, the open gap-ledger items, the operator decisions, the host caveat — is unchanged from rev 26 and restated as-is; only committed-vs-pushed state and the live anchors move.

Push ordering is a hard constraint, not a preference. submodules/helix_agent (9 ahead) and submodules/helix_llm (3 ahead) **MUST** be pushed to all their upstreams BEFORE meta main. Meta commit e47c8a30 bumps both gitlinks; if meta is pushed first, those gitlinks point at commits no remote has, and every fresh clone fails `git submodule update` with `not our ref`. Push submodules, verify with `git ls-remote`, then push meta. All pushes are fast-forward only — force-push is forbidden without exception (§11.4.113).

Rev 25 superseded rev 24 (2026-09-05T18:37:41Z) for live state and both resume prompts. Rev 24 superseded rev 23 for push status. Sections 1-7 below the “END REV 25 BLOCK” marker are rev 23’s forensic record of the 2026-09-03 session, preserved as history; **this pass did not re-verify them**. Scope of this pass was docs/CONTINUATION.md, this file and their .html/.pdf export siblings. docs/guides/cli_agent_integration/**, scripts/systemd/ and the submodules/ worktrees are being written by another live stream right now and were **read but not written** by this pass.

Contents

- [Read this first — the one-paragraph state](#)
- [Live anchors — MEASURED this pass](#)
- [0 · Resume prompt — SHORT form](#)

- [0b · Resume prompt — FULL form](#)
- [What the batch landed \(now committed\)](#)
- [What is still open](#)
- [Operator decisions in force](#)
- [Host caveat and environment quirks](#)
- [1 · Start here](#) *(rev 23 history)*
- [2 · What is actually happening](#) *(rev 23 history)*
- [2b · How to actually run the system \(verified 2026-09-03\)](#) *(rev 23 history)*
- [2b-2 · HelixLLM, HelixAgent, and the Claude Toolkit sync \(verified 2026-09-03\)](#) *(rev 23 history)*
- [2c · Full retest results \(2026-09-03, and how to read them\)](#) *(rev 23 history)*
- [3 · What was fixed in the 2026-09-02/03 session](#) *(rev 23 history)*
- [4 · Behaviour changes a user could notice](#) *(rev 23 history)*
- [5 · Open, and needing a decision rather than more work](#) *(rev 23 history)*
- [6 · House rules that bit someone this session](#) *(rev 23 history)*
- [7 · Resume prompt \(superseded — see \\$0/\\$0b at the top of this file\)](#) *(rev 23 history)*

Read this first — the one-paragraph state

The large cloud-gate + CLI-agent-fanout batch is **committed and NOT pushed**. It landed as 15 commits on meta main (926eaf3e..dfb06d38) plus f3f0540b. **The next action is the push, and its ORDER is a hard constraint: both submodules first, meta last** — e47c8a30 bumps the gitlinks, so pushing meta first leaves them pointing at commits no remote has. Pushes are fast-forward only; force-push is forbidden without exception (§11.4.113). At least one other stream is still writing this checkout (docs/guides/cli_agent_integration/**, scripts/systemd/, the submodules/ worktrees) — do not sweep its files into a push-prep commit. Twelve gap-ledger entries remain open, several deliberately; nothing below marks them done.

Live anchors — MEASURED this pass

Re-derived by execution at 2026-09-07T10:02:04Z (§11.4.6 — nothing carried forward from rev 26). **Re-derive them yourself before relying on them.**

repo	branch	HEAD	ahead	behind	worktree
meta (checkout root /home/ milosvasic/ Projects/ helix_code)	main	f3f0540b	23	0	changed entries, count not quoted here — another

repo	branch	HEAD	ahead	behind	worktree
					stream is still writing this tree (§11.4.6); re-derive with <code>git status --porcelain, piped to wc -l</code>
submodules/helix_llm	main	b25641f	3	not checked	not checked
submodules/helix_agent	main	ca24b2fd	9	not checked	dirty — another stream owns it

- **35 commits unpushed across three repos** at the moment of measurement; **36** once the rev 27 commit below is counted.
- These anchors were measured **immediately before the rev 27 commit itself**. Rev 27 adds one commit to meta main, so on a fresh read expect meta HEAD to be the rev 27 commit and the meta ahead-count to be **24**, not 23. The submodule anchors are unaffected. Re-derive rather than trust either number.
- Meta HEAD commit subject: *“test(regression): classify EADDRINUSE so a saturated host SKIPS, not FAILs”*.
- **Push order: helix_agent and helix_llm FIRST, meta LAST**. Meta e47c8a30 bumps both gitlinks to ca24b2fd / b25641f; pushing meta first publishes gitlinks no remote can resolve. Verify each submodule with `git ls-remote <remote> main` before pushing meta.
- Four configured remotes — origin, github, gitlab, upstream — all report the same 23-ahead because they resolve to **two** physical hosts: `git@github.com:HelixDevelopment/code.git` and `git@gitlab.com:helixdevelopment1/HelixCode.git`.
- `git rev-list --count HEAD..@{u} = 0` — nothing to pull on meta main.
- The docs/qa/phase1_fullhttp_e2e_* evidence directories are now **tracked** — they were committed in 27a5622c per the operator decision. No count is quoted here; re-derive live: `git ls-files docs/qa/ | grep -o 'phase1_fullhttp_e2e_[^/]*' | sort -u | wc -l` for tracked, `git status --porcelain | grep -c '^?? docs/qa/'` for any that arrived since. The operator brief and rev 24 both said 33 — already superseded by

measurement once; treat every prior count, including any number that was ever written in this section, as stale on sight.

- **In flight right now:** the batch's own streams and reviewers have finished — that is why the batch is committed. **At least one OTHER stream is still live** and owns docs/guides/cli_agent_integration/**, scripts/systemd/helixllm-coder-native.service, the submodules/helix_agent and submodules/helix_llm worktrees, and three newer docs/qa/ directories (2026-09-07-helix-models-toolkit, agent_config_fanout_fixes_*, cli_agent_model_usability_*). All were dirty at 2026-09-07T10:02:04Z. **Do not write, stage or push-prepare those paths** until you have confirmed that stream has finished (\$11.4.84 / \$11.4.119).

0 · Resume prompt — SHORT form

Read `RESUME.md` (this file, the rev 27 block at the top) and `docs/CONTINUATION.md` (rev 27, section ## 2026-09-06 – cloud-gate + CLI-agent-fanout batch), run `git fetch --all --prune --tags`, then push the committed batch — the batch IS COMMITTED and NOT PUSHED, meta main is 23 ahead at `f3f0540b` with `helix_agent` 9 ahead at `ca24b2fd` and `helix_llm` 3 ahead at `b25641f` (35 unpushed), and **the push order is a hard constraint: both submodules FIRST (verify each with `git ls-remote`), meta LAST**, because meta `e47c8a30` bumps those gitlinks and publishing them before the submodule commits exist breaks every fresh clone; all pushes are fast-forward only and force-push is forbidden without exception (§11.4.113). Twelve gap-ledger entries `HXC-002-F3-01..-12` are still open. At least one other stream is still writing this checkout — `docs/guides/cli_agent_integration/**`, `scripts/systemd/` and the `submodules/` `worktrees` are dirty and are NOT yours; do not stage them. This host is at ~3.5x CPU oversubscription from a foreign workload so every timing-sensitive result is suspect.

0b · Resume prompt — FULL form

Read RESUME.md (the rev 27 block at the top of this file), then docs/CONTINUATION.md rev 27 (metadata table + the section "## 2026-09-06 – cloud-gate + CLI-agent-fanout batch"). Then run these and READ THE OUTPUT before acting – §11.4.6, do not trust any number below without re-deriving it, all of them are from 2026-09-07T10:02:04Z:

```
git fetch --all --prune --tags
git rev-list --count HEAD..@{u} # expect 0 (not
git rev-list --count @{u}..HEAD # expect 23, HE
git -C submodules/helix_agent rev-list --count @{u}..HEAD # expect 9, H
git -C submodules/helix_llm rev-list --count @{u}..HEAD # expect 3, H
git status --porcelain | wc -l # re-derive liv
# stream is s
# writing thi
```

so no count
survives to
quoted here
(§11.4.6)

STATE: the cloud-gate + CLI-agent-fanout batch IS COMMITTED and IS NOT PUSHED. It landed as 15 commits on meta main (926eaf3e..dfb06d38) plus f3f0540b (a determinism guard that arrived after its group had already been committed). The batch reached a clean GO before commit per §11.4.134.

THE NEXT ACTION IS THE PUSH, AND ITS ORDER IS A HARD CONSTRAINT:

1. push submodules/helix_agent (9 ahead, ca24b2fd) to ALL its upstreams
2. push submodules/helix_llm (3 ahead, b25641f) to ALL its upstreams
3. verify BOTH with: `git ls-remote <remote> main`
4. only THEN push meta main (23 ahead, f3f0540b) to all upstreams

Meta commit e47c8a30 bumps both gitlinks. Pushing meta before the submodule commits exist on the remotes publishes gitlinks nothing can resolve, and every fresh clone then fails `'git submodule update'` with "not our ref". All pushes are FAST-FORWARD ONLY. Force-push is forbidden without exception, with no operator-approval path (§11.4.113); if a remote rejects fetch and MERGE onto the latest remote tip, then push again.

CONCURRENCY: at least one other stream is STILL writing this checkout. It owns docs/guides/cli_agent_integration/**, scripts/systemd/ and the submodules/ worktrees – all dirty right now, none of them yours. Do not stage or push-prepare those paths (§11.4.84 working-tree quiescence, §11.4.119 single-resource-owner, §11.4.176 exactly-once claim). Note the submodule WORKTREES being dirty does not block the push: the gitlink commits to be pushed are already made.

WHAT THE BATCH LANDED (now committed):

- The llm.cloud.enabled gate is closed at EVERY constructor path. KoboldAI was exempt-by-identity while shipping a bearer credential; NewProvider was an ungated back door. A filepath.WalkDir AST scan now enumerates ErrCloudDisabled sites so a new one cannot hide.
- Credential redaction: FOUR distinct carriers were found across rounds the base URL; `*url.Error.URL`; the wrapped cause, where `net/url` parses scheme-less credential AS the scheme; and `url.Redacted()` preserving the USERNAME (the documented Stripe key:@host form, where the credential IS the username). Now 5 sites x 14 shapes = 70 subtests, with an AST cross-check between two shape tables so a shape declared in one cannot be absent from the other.
- A typed-nil interface defect – a non-nil Provider wrapping a nil pointer on every gate refusal – closed across ~31 factory arms, with a static AST scan so a fourth occurrence cannot be written.
- HTTP status semantics: a closed cloud gate returned a retryable 503; now 403 caller-sourced / 500 server-sourced, provenance tracked.
- The CLI-agent installer --dry-run redactor took five rounds, ending in a POSITIVE ALLOWLIST implemented in awk, because "mask unless allowlisted" is not expressible as an ERE – expressing it as a negative

- character class WAS the defect.
- The fan-out build gate: FIFTEEN fail-opens found and closed. It now carries a BASH EXECUTION ORACLE – 17 fixtures run under real bash with an instrumented stub, so bash, not a regex, decides mention-vs-call.
 - Determinism: 6 non-reproducible tests fixed and proven at -count=5 under synthetic load; 39 env/global leak sites closed; server.New()'s process-global cloud-gate write isolated in tests WITHOUT removing the production write – removing it would have been a silent runtime regression, because cmd/server/main.go has zero occurrences of "cloud"
 - submodules/helix_agent: a "cloud opt-in" gated ONE of FOUR cloud paths. The four are env-key providers, the startup verifier (which sends real prompts at boot), zen discovery, and embeddings to api.openai.com. All four now run through one predicate in a new internal/localfirst leaf package – an import cycle had forced a duplicated predicate and the duplicate had drifted.

WHAT IS STILL OPEN – carry these, do not let them evaporate:

1. Twelve gap-ledger entries HXC-002-F3-01 .. HXC-002-F3-12 in helix_code/docs/qa/2026-09-05-gap-ledger.md. Several are open DELIBERATELY: a bounded, visible known gap beats a rushed fix that trades it for an unbounded new one – which was this batch's recurring failure mode, each round's fix creating the next round's finding.
2. HXC-002-F3-09 – FIX DIRECTION CONFIRMED BY MEASUREMENT. A live E2E test asserts the coder emits structured tool_calls. Measured today: the CODER DOES NOT (finish_reason "stop", the call arriving as a fenced JSON blob in content) and the GATEWAY DOES (finish_reason "tool_calls", tool_calls present) at token budgets 16/32/64/200. So point the test at the gateway. Keep a negative guard pinning the coder's real contract so a future coder that gains tool_calls fails loudly rather than silently changing the contract.
3. Structural fail-opens the build gate cannot see – a dead function, `if false`, `[ ] && ...` – plus 8 measured false refusals. All are declared in-code; none is a live exploit.
4. gin.SetMode() with no restore: 62 occurrences measured module-wide. Latent order-dependence. (The brief said 51; 62 is measured.)
5. time.Sleep-as-synchronisation: 455 occurrences measured module-wide, 3 packages being addressed now. (The brief said ~155 on a narrower scope.) Neither figure separates genuine sleeps from sleeps used as synchronisation – that split is UNMEASURED.

OPERATOR DECISIONS IN FORCE:

- The untracked docs/qa/phase1_fullhttp_e2e_* directories (count not quoted here – the tree is being written by concurrent streams; re-derive with `git status --porcelain | grep -c '^?? docs/qa/phase1_fullhttp_e2e_') WILL be committed.
- Submodule pushes WAIT until helix_agent clears review.

WHEN YOU EVENTUALLY PUSH: fast-forward only, NEVER force, no exception (§11.4.113). Integrate by merging onto each remote's latest main.

HOST CAVEAT: this machine runs at roughly 3.5x CPU oversubscription from a FOREIGN workload that MUST NOT be touched. Every timing-sensitive

validation here is suspect – the -count=5 determinism proofs above were obtained under exactly this contention, which strengthens a PASS but makes a FAIL ambiguous. Keep a small process footprint and REAP every child.

ENVIRONMENT QUIRKS (this session; re-verify):

- A Semgrep PreToolUse hook REJECTS any Bash command string containing the Go toolchain token, even quoted. Put such commands in a script FILE and bash the file; prefer `make` targets in helix_code where one exists
- The Edit tool is unreliable here. Use Write plus assertion-guarded python3 replace scripts, and READ THE TARGET BACK afterwards to confirm the change actually landed before trusting it.

What the batch landed (now committed)

Recorded in full, with per-claim MEASURED / INHERITED tagging, in docs/CONTINUATION.md rev 25 under ## 2026-09-06 – cloud-gate + CLI-agent-fanout batch. Summary: the cloud gate closed at every constructor path (KoboldAI, NewProvider); four credential carriers redacted behind 70 subtests with an AST cross-check; a typed-nil defect closed across ~31 factory arms with a static AST scan; 503 → 403/500 status semantics; an awk positive-allowlist - -dry-run redactor; fifteen fan-out-gate fail-opens closed behind a bash execution oracle; six determinism fixes and 39 env-leak sites; and helix_agent's four cloud paths unified behind a single internal/localfirst predicate.

Spot-verified by execution during this pass (§11.4.6): the gate is present at helix_code/internal/llm/koboldai_provider.go:160 and at helix_code/internal/llm/factory.go:113 inside NewProvider (declared :92); ErrCloudDisabled has 86 references across 20+ files; three AST-driven guard tests exist; internal/server/llm_generate.go carries StatusInternalServerError at :691, StatusForbidden at :694, residual StatusServiceUnavailable at :699; scripts/gates/agent_config_fanout_gate.sh is present, executable, and untracked (byte size not quoted here — an untracked file's size can change while parallel streams write; re-derive with `wc -c scripts/gates/agent_config_fanout_gate.sh`); and submodules/helix_agent/internal/localfirst/{localfirst.go,localfirst_test.go} exists. Everything else in the summary is **INHERITED** from the batch's own streams and was **not** re-verified here.

What is still open

See the FULL resume prompt above, items 1-5, and the matching section in docs/CONTINUATION.md rev 25. In one line each: twelve gap-ledger entries HXC-002-F3-01..-12 (several open deliberately); HXC-002-F3-09 with a measurement-confirmed fix direction (point the test at the gateway); structural fail-opens the

build gate cannot see plus 8 measured false refusals; 62 measured
gin.SetMode() sites with no restore; 455 measured time.Sleep sites.

Operator decisions in force

1. The untracked docs/qa/phase1_fullhttp_e2e_* directories **WILL be committed** (count not quoted here — the working tree is being written by concurrent streams, so any committed number is false on arrival, §11.4.6; re-derive with `git status --porcelain | grep -c '^?? docs/qa/phase1_fullhttp_e2e_'`).
2. **Submodule pushes wait** until helix_agent clears review.

Host caveat and environment quirks

The host runs at roughly **3.5x CPU oversubscription from a FOREIGN workload that must not be touched**. Timing-sensitive results obtained here — including the `-count=5` determinism proofs — carry that contention as context: it strengthens a PASS and makes a FAIL ambiguous. Keep a small process footprint and reap every child.

A **Semgrep PreToolUse hook rejects** any Bash command string containing the Go toolchain token; put such commands in a script file and bash it. The **Edit tool is unreliable** this session — use Write plus assertion-guarded python3 replace scripts and read the target file back to confirm the write landed.

END REV 25 BLOCK. Everything from here down is rev 23's 2026-09-03 forensic record, preserved as history and **not** independently re-verified by this pass or by rev 24. Where a claim below conflicts with the rev 25 block above (push status, HEAD SHAs, suite state, “what is actually happening”), **the rev 25 block above is authoritative**.

Rev 23 · 2026-09-03 ~14:00 CEST. Supersedes rev 22 (same morning), rev 21 and rev 20 (2026-08-14).

What changed since rev 22, and it is the headline: EVERYTHING IS NOW PUSHED, and the system BOOTS AND SERVES. [STALE as of rev 24 — see the block above. 18 commits are unpushed as of 2026-09-05T18:37:41Z.] Rev 22 said “Nothing is pushed”; that is no longer true and §2 below is corrected. Three defects that each independently stopped the stack from starting were found by actually running it, and fixed.

Rev 20 was three weeks stale and **its first command did not work**: it said to `cd /home/milos/Factory/projects/tools_and_research/helix_code`, which

does not exist on this host. docs/CONTINUATION.md's own TL;DR names a third, also non-existent path (/run/media/milosvasic/DATA4TB/Projects/HelixCode). The real checkout is below. Rev 20 also reported "every repository is clean and pushed" and declared no active programme; neither is true now.

Governing rule (kept verbatim from rev 20 — it has earned its place). Every count, list, hash and stream-state below is a **snapshot**. Re-derive before acting on any of it; the commands are inline. Counts here have moved under scrutiny repeatedly.

This revision has a stronger reason than usual. It was written while **six subagents were committing concurrently** to submodules/helix_llm, helix_code/internal/config and the Claude Toolkit. Every hash and count below was already moving as it was recorded. Treat all of it as "roughly here", never as current.

Instrument warning (kept from rev 20, measured 2026-08-14). grep is a **shell function** in the agent shell (ugrep with --ignore-files --hidden) and **silently skips gitignored paths**: bare grep -r found 1 hit where /usr/bin/grep -r found 2. It is not exported, so bash script.sh children get real GNU grep. **Use /usr/bin/grep for every inline count** and say which instrument produced each number.

1 • Start here

```
cd /home/milosvasic/Projects/helix_code      # the ONLY correct
      path; three docs said otherwise
git fetch --all --prune
git log --oneline -1                          # rev 21 was written
      at 7d45aed4; rev 24 at 2372d7bf
git status --porcelain | wc -l               # 12 when rev 23
      written, 47 when rev 24 written, and moving
git -C submodules/helix_llm log --oneline -1
      # 6e278e8 when rev 23 written; b25641f when rev 24
      written
```

Read .remember/now.md first, then this file, then specs/002-adaptive-local-model-serving/progress.yml — that last one is the real record of what is broken and what has been proven about it. [Rev 24 note: also check whether docs/superpowers/plans/2026-09-05-local-adaptive-serving.md (plan task W2c-1) supersedes or complements this spec — not reconciled by the rev 24 pass.]

2 · What is actually happening

There IS an active programme. Rev 20 and docs/CONTINUATION.md both say there is not. That is the single most misleading thing in the older docs.

Feature **002 adaptive-local-model-serving** is mid-execution: specs/002-adaptive-local-model-serving/.

READ THIS FIRST: the naming/export batch is NO-GO. An independent re-review confirmed four findings BY RUNNING CODE, and THREE OF THEM WERE CREATED BY THE FIXES. Two are user-facing and serious: a user holding a pre-fix helixllm-127-0-0-1-... identifier is now silently answered by the WRONG MODEL (the original CRITICAL, reproduced for the population the batch itself created), and - -apply DELETES a user's entire configuration for a host that is merely restarting. Fixes are in flight. Do not treat this batch as done, and do not trust an earlier summary that says the fixes landed cleanly — they landed, and they each created something new.

- **89 of 97 tasks** complete (/usr/bin/grep -c '^-[x\]' .../tasks.md).
- **93 findings** recorded in progress.yml. Roughly 20 open, and the open ones are now mostly decisions rather than unfinished work — see §5.
- ~~Everything is pushed~~ (2026-09-03 ~13:45 CEST), ~~fast forward, no force.~~
STALE — corrected by rev 24 above. As of 2026-09-05T18:37:41Z, 18 commits are unpushed across the meta repo (7), helix_llm (3) and helix_agent (8). The table immediately below reflects the state at the time rev 23 was written (2026-09-03) and is preserved for history only — do not treat any SHA in it as current:

repo	HEAD (as of rev 23, 2026-09-03 — STALE)	upstreams
meta	b752a807	GitHub Helix-CLI, GitLab HelixCode
helix_llm	1efda3b5	GitHub, GitLab
helix_agent	64cf8921	HelixDevelopment/HelixAgent, vasic-digital/HelixAgent
claude_toolkit	328cf27b	GitFlic, GitHub, GitLab, GitVerse (branch fix/helixllm-export-

repo	HEAD (as of rev 23, 2026-09-03 — STALE)	upstreams
		review-findings, NOT merged to main)

Two upstreams reported a **move**: HelixLLM -> HelixDevelopment/llm.git and Helix-CLI -> HelixDevelopment/code.git. Pushes still succeed via the redirect; the configured URLs are stale and worth updating. GitHub also reports **81 Dependabot vulnerabilities** on vasic-digital/HelixAgent (1 critical, 56 high, 21 moderate, 3 low).

The 8 open tasks are NOT stalled work. Four (T037, T055, T068, T084) are [REVIEW] tasks that **already ran and returned findings**; §11.4.134 requires iterating to a zero-finding GO, so they stay open until the fixes land and a re-review comes back clean. T052/T053 need a running system. T054 is an outward-facing publish and is explicitly an operator checkpoint. T097 is the final review.

2b · How to actually run the system (verified 2026-09-03)

The containerised route (`./helix start`) does NOT work yet — see B00T-4. Use the native route, which is the one Rule 4 names first (`make build -> ./bin/<app>`). The server auto-boots its own Postgres and Redis in podman per §11.4.76, so no compose file is needed at all.

```
cd /home/milosvasic/Projects/helix_code

# .env is gitignored; generate the three REQUIRED secrets if
# absent
cp .env.example .env && chmod 600 .env
for k in HELIX_DATABASE_PASSWORD HELIX_REDIS_PASSWORD
    HELIX_AUTH_JWT_SECRET; do
    sed -i "s|^$k=.*|$k=$(openssl rand -hex 32)|" .env
done

cd helix_code && go build -o bin/helixcode ./cmd/server
set -a; . ../.env; set +a
HELIX_REDIS_HOST=localhost ./bin/helixcode
```

The server REFUSES to start if a secret is missing, naming the variable — that is deliberate, and it is the placeholder guard added earlier in this programme:

```
config validation failed: auth.jwt_secret is still the unexpanded
placeholder for ${HELIX_AUTH_JWT_SECRET} ... Export it and start again.
```

On success it prints `Infra auto-boot: podman booted postgres:<p>redis:<p>` and listens on `:8080`. Two containers appear, `helixcode-autoboot-postgres` and `helixcode-autoboot-redis`. Opt out with `HELIX_AUTOBOOT_INFRA=false` to use external infra instead.

Do not run the test suites while the server is up. Verified this session: a live server on 8080 silently changes outcomes in BOTH directions — it wakes the otherwise-vacuous tests/memory suite (TEST-2) and breaks a challenge asserting a CLOSED port reports SKIPPED. See METHOD-1.

Verified working end-to-end:

probe	result
GET /health	200 { "status": "healthy", "version": "1.0.0" }
GET /api/v1/server/info	200, database.connected=true
GET /api/v1/llm/providers	200, 8 providers
GET /api/v1/llm/models	200, 8 models
GET /api/v1/memory/systems	200, 6 systems
GET /api/v1/metrics	200, live pool 6 active / 6 idle / 20 max
POST /api/v1/auth/register	201, user persisted with a real UUID
POST /api/v1/auth/login	200, 287-char JWT
/tasks/workers/system/stats	401 without a token, 200 with one
GET /api/v1/auth/me	404 despite being registered — OBS-1

2b-2 · HelixLLM, HelixAgent, and the Claude Toolkit sync (verified 2026-09-03)

HelixLLM — port 8443, HTTPS, self-signed

```
cd submodules/helix_llm
go build -o bin/helixllm ./cmd/helixllm
HELIX_MODE=full ./bin/helixllm # listens on :8443
curl -sk https://127.0.0.1:8443/v1/models # -k, or use the CA
bundle below
```

Plain HTTP is refused with 400 Client sent an HTTP request to an HTTPS server — that 400 is the TLS mismatch, not a broken endpoint.

CHECK THE RUNNING BINARY'S AGE BEFORE TRUSTING ANY PROBE. A helixllm that had been up for 16 hours was still serving {"object": "list", "data": null} — the defect ab34fa8 had ALREADY fixed in source. Source-green said nothing about what was serving (§11.4.108 SOURCE→RUNTIME). Compare `ps -o lstart= -p <pid>` against `git log -1`, and rebuild before believing a live result.

The Claude Toolkit sync — two gotchas that will cost you an hour

```
export CURL_CA_BUNDLE=$PWD/submodules/helix_llm/certs/cert.pem
claude-providers helixllm-export --host https://localhost:8443/v1
```

1. **The base URL must already contain /v1.** `_cma_helixllm_fetch_models` appends `/models`, so `--host https://...:8443` requests `/models` and misses.
2. **The self-signed cert needs a trust anchor.** The fetcher uses a bare `curl -sf` with no `--cacert`, so it fails with **curl exit 60** and the tool reports only *“host ... did not answer with a model listing”* — which reads like the host is down when it is answering perfectly. `CURL_CA_BUNDLE` fixes it with no code change; the cert covers `localhost`, `127.0.0.1`, `192.168.0.241`.

What it exports today: nothing, correctly. HelixLLM catalogues exactly one model, `helixllm/anton/Llama-3.1-70B-Instruct-Q4_K_M`, as availability: withheld / provider_unavailable, and the toolkit refuses to advertise a model that is not actually being served. `anton` IS this host, there are no GGUF weights on it, and no `llama.cpp` server is running. Note also that `HELIX_LLM_LOCAL_MODEL` defaults to that 70B Q4_K_M (~40 GB) on a box with **30 GB RAM and 12 GB VRAM** — it could not run it even with the weights.

To actually get a usable model into Claude Code you need a served model: place a GGUF this host can run (RTX 3060 / 12 GB — an 8B Q4_K_M at ~4.9 GB fits comfortably), point `HELIX_LLM_LOCAL_MODEL` at it, run `llama-server` (`/usr/bin/llama-server` is installed; `ollama` is NOT), then re-run the export.

HelixAgent — boots its whole stack on podman, needs one secret

```
cd submodules/helix_agent
go build -o bin/helixagent ./cmd/helixagent
printf 'JWT_SECRET=%s\n' "$(openssl rand -hex 32)" >> .env &&
    chmod 600 .env
set -a; . ./env; set +a && ./bin/helixagent
```

Observed on a real run: Container adapter initialized via Containers module runtime=podman, postgres+redis+chromadb started via podman-compose with all three health checks PASSED, 1163 skills across 16 categories, 32 MCP servers, liveness probe on :8111 — then it stops with Failed to initialize auth middleware: JWT secret key is required. That is the only blocker; .env is gitignored there (.gitignore lines 3/38/51).

Its LLMsVerifier pipeline also completes with **0 providers discovered** out of 44 candidate env vars — HelixAgent exposes no models because no provider keys are in ITS environment. The operator’s keys live in ~/api_keys.sh, which the toolkit reads and helixagent does not.

2c · Full retest results (2026-09-03, and how to read them)

repo	result
helix_llm	exit 0 — 54 packages ok, 0 FAIL
helix_agent	exit 0 — 289 packages ok, 0 FAIL
helix_code	exit 1 — 192 ok, 7 failing packages (below)
claude_toolkit	exit 1 — 2279 assertions pass, 12 fail = 2 root causes

helix_code’s 7 failures are **3 real and 4 artefacts of the sweep itself**:

- **3 GUI packages** (applications/{desktop,aurora_os,harmony_os}) fail to BUILD. Genuine environment gap: this host has no OpenGL/X11 development headers, so the Fyne -> go-gl -> glfw chain cannot compile (gl.pc and X11/Xlib.h both absent). Not a code defect. make desktop-nogui exists precisely for this. Server and CLI build fine.
- **4 timing / connection-pool packages** fail inside go test ./... and PASS when run alone:

package	in the parallel sweep	alone, quiet host
internal/providers/httpclient	FAIL	ok 0.015s
tests/performance/scenarios	FAIL	ok 3.980s

package	in the parallel sweep	alone, quiet host
tests/regression	FAIL	ok 1.792s
tests/memory	FAIL	ok 98.991s (all 15 tests)

Why, and it is structural. `go test ./...` runs package binaries in parallel up to GOMAXPROCS. Measured on this host: the `helix_code` sweep alone drove load to **70 on 16 CPUs**. Every one of those four measures something load-sensitive — wall-clock stability, HTTP keep-alive pool reuse, post-GC live heap. They cannot be measured reliably inside their own sweep.

So the honest reading is: **the suite is green apart from a documented toolchain gap**, but the timing-sensitive packages need `-p 1` or a separate target. Do not “fix” them by loosening their bounds — that is the \$11.4.120 forbidden move, and their bounds are what make them worth having.

There is a good precedent already in the tree: `helix_llm`’s `internal/testing` DETECTS the contention and skips that step with a reason (“host too loaded to measure concurrency ... re-run on a quieter host”). Its only flaw is that the wrapper asserts “passed” rather than accepting “skipped”. That pattern is worth generalising.

[Rev 24 note: this §2c table describes the 2026-09-03 suite. The rev 24 block above records a DIFFERENT, more current suite check — make `verify-compile` GREEN plus a targeted package run — for the 2026-09-05 W2c-1 batch. The two are not directly comparable; do not assume this table’s numbers still hold.]

3 · What was fixed in the 2026-09-02/03 session

Every one of these was **reproduced before being fixed**, and each carries a paired mutation that was `diff`-verified as actually applied.

What was wrong	Where
Published model identifiers were decorative. <code>ResolveModelName</code> was tested but never on the request path; <code>fallback.Chain</code> overwrote <code>req.Model</code> with each provider’s FIRST model. A client asking for a <code>gpu-01</code> identifier was answered by chutes with <code>deepseek-chat</code> .	<code>helix_llm a5a2eb9</code>

What was wrong	Where
Selection never read VRAM — it checked whether accelerators <i>existed</i> , then compared requirements against host RAM. ResourceAccelerator was declared in <code>option.go</code> and referenced nowhere else in the repo.	helix_llm 39701cd
The VRAM broker parsed “the FIRST GPU row” and a test asserted that as correct. On a two-GPU host it refused a 6 GiB request while the other card had 12689 MiB free.	helix_llm 39701cd
The attestation proof was HMAC(secret, nonce) — it proved someone held the secret, not <i>who answered</i> . A relay could borrow a genuine instance’s answer to our own nonce. Reproduced with the user’s prompt, an opened file and an upstream credential arriving at a hostile host.	helix_llm 3efc367
The binding was established on the probe’s connection but Send re-resolved the name, so DNS could move between the two requests.	helix_llm 6e278e8
Unexpanded <code>\${...}</code> used verbatim as a credential. With the variable unset, every JWT was signed with a string committed to this repository.	helix_llm 5b68ffa, meta ba7f6133
<code>redis.db</code> was mis-tagged database, so every deployment used Redis DB 0 whatever the operator configured.	meta ba7f6133
The videogen lane could plan a build the service refuses to load (two precision lists disagreed).	helix_llm 2167525

Landed later the same morning:

What was wrong	Where
The HelixCode and OpenCode exporters had NO CALLER — those	helix_llm f63b96f

What was wrong	Where
artifacts were unobtainable by any user. Git history showed never-completed wiring, not rot, so completing was right.	
The published identity named 127.0.0.1 on every machine, so it named no machine AND collided across machines; the toolkit's group_by then silently dropped the second host.	helix_llm f63b96f
Selection and the broker disagreed about headroom: a 10 GiB model on an 11.5 GiB card was offered and then refused. Two placements could also each be told the same card was free.	helix_llm 087947d
agentgen-boot took its model from configuration and its VRAM figure from a SECOND env var a human had to keep in step. Naming a 19.5 GiB model with the figure untouched gave ADMIT-OK and exit 0.	helix_llm 0c43b11
A listing of nothing returned "data": null with no reason — a body that reads as malformed, from the branch next to the one documenting that exact rule.	helix_llm ab34fa8
A request cancelled BEFORE it started could still complete: select raced an already-ready ctx.Done() against an already-ready response, and Go picks at random. 39/40 before, 40/40 after.	helix_llm 6fba621
An unpaired bracket in a configured host silently discarded a NAMED PRODUCTION HOST and connected to loopback (db.prod.internal[::1 → ::1).	helix_agent be54764d
production-config.yaml was not valid YAML — an AI assistant's transcript had been pasted into it —	meta 4e2d742c

What was wrong	Where
and beneath that, 210 lines YAML was already discarding.	
The notifications: block reached nothing; and the expander used os.Expand, which eats \$\$, so an SMTP password pa\$\$word became pa.	meta 721f6c6e
A whitespace-only credential validated cleanly and then rejected every request including the correct one.	helix_llm ad813ef

Two lessons worth carrying forward, both recorded in progress.yml:

- *A fix that breaks a sibling gate is not automatically a stale gate.* The placeholder fix segfaulted on a nil optional section; its own tests missed it because their fixture populated that section, and a bcrypt guard caught it. Reconciling the gate instead of investigating would have shipped a crash.
- *“No output” is ambiguous between “clean” and “broken”.* This bit FIVE times, including twice against me: a pgrep waiter that matched itself, a mutation gofmt silently prevented from applying, a crashed grep, a test log I had filtered myself and then counted, and a rejection message my own filter cut off. Always assert the check ran.
- *A flaky test and a real race look identical from the failure rate.* TestLSPClient_ContextCancellation was written off as flaky by two separate agents. The question that settled it was not how often it failed or whether it passed in isolation — both were true — but whether the scenario contained any LEGITIMATE timing. A context cancelled before the call contains none, so a random outcome could only be a defect.
- *A review finding is a place to look, not a thing to implement.* Three severity claims from one review did not survive verification, and in each case the fix that shipped differs from the fix the report implied. Two premises of MY OWN also turned out false and were caught by the agents I gave them to.

4 • Behaviour changes a user could notice

Not defects — deliberate, and worth knowing before someone reports them as bugs.

1. **Naming a model now pins it.** gpt-4o no longer cross-falls-back to another provider on failure. A named model is a choice, not a hint.

2. **A re-addressed instance needs a fresh Discover.** Send dials the address that authenticated, so a DHCP or container-restart address change is not followed until rediscovery. The alternative — re-resolving — is the hole.
 3. **Startup refuses an unexpanded `${...}` credential.** A deployment relying on the old silent behaviour will now fail to start, with the field and the variable named in the error.
-

5 · Open, and needing a decision rather than more work

- **Selection uses zero VRAM headroom; the broker reserves 2 GiB.** A 10 GiB model on an 11.5 GiB card is offered and then refused — the user is shown something that cannot start. A seam exists (`Reserve.AcceleratorHeadroomBytes`) and nothing sets it. OPEN-17.
- **Concurrent placements do not draw down device memory.** Commitment has no accelerator dimension, so two placements can each be told the same card is free. OPEN-18.
- **Two integration model-listing tests fail at baseline** and three agents each independently proved the failures pre-date their work. Believed to need live services — but that is the assumption, not a finding. OPEN-19.
- **The shipped config writes `${...}` into fields nothing expands,** so a provider endpoint is a literal that is not a URL. OPEN-15.
- **13 credentials remain exposed and unrotated.** The operator's standing decision is "just keep the record" — do not add rotation tooling; keep the documented list current.
- **The agent lane's three former candidates are not in the catalogue,** so an operator serving Mistral-Nemo-2407, GLM-4.7-Flash or DeepSeek-Coder-V2-Lite through it will now be REFUSED. That is the flip side of the FR-056 fix: those three never had a measured footprint, which is exactly what the fixed 9 GiB placeholder stood in for. Resolve by measuring them on a host that HAS the GGUFs and adding catalogue entries with recorded provenance — or by accepting the narrower set. Do not estimate the figures. OPEN-24.
- **Five of six text catalogue entries carry `requires_accelerator: false`,** so selection skips the device axis and a host-RAM figure reaches a VRAM broker. The CRITICAL-5 shape in a narrower form; the vision lane has it too. OPEN-25.
- **The containerised boot is still broken.** `helix_code/go.mod` has 43 `replace` directives pointing at `../submodules/*`, outside the directory the Dockerfile copies, so `go mod download` fails inside the image. The targets total 4.2 GB (`helix_qa` 2.5 GB, `helix_agent` 2.0 GB, mostly vendored trees), so the fix needs a layout change plus a scoped `.dockerignore`. The native route works and needs no `compose` — see §2b. BOOT-4.

- **The whole tests/memory suite has been silently vacuous.** Every test in it probes /health first and t. Skips when nothing answers, so with no server up the package reports ok . . . 0.077s having exercised nothing. With a server up it runs for 23s and the concurrent-request test produced a rising live-heap signal (signed-R2 0.6565, rise 51.57% of mean). That signal is NOT yet a defect — it was taken at load 44, the heap is under 1 MB, and the shape is flat for 7 waves then a late jump, as consistent with transport idle-pool growth as with a leak. Needs a quiet-host re-run. The coverage gap is the real finding. TEST-2.
 - **A Claude Toolkit test asserts against the operator's live \$HOME.** scripts/tests/test_claude.sh:7 hardcodes ALIAS_FILE="\$HOME/.local/share/claude-multi-account/aliases.sh" and sources only lib/assert.sh, never lib/sandbox.sh. It FAILs xiaomi uses cma_run_provider purely because the operator has no xiaomi provider configured. Same class as helix_agent's 913d1f02 ("validate the config this repo ships, not the operator's").
 - **internal/lifecycle's concurrent-evict test fails under CPU contention,** on its own LIVENESS precondition rather than the invariant it guards. Unlike the LSP case this scenario does contain legitimate timing, so it is a genuine flake — but it should retry or SKIP with a reason rather than FAIL and imply the invariant broke. It has already cost two agents time. OPEN-23.
-

6 • House rules that bit someone this session

- **Staging by path is NOT enough.** Use `git commit -- <paths>`. Multiple agents share this checkout, and `git add <path>` adds to the index while `git commit` commits the WHOLE index — including whatever another agent staged before you got there. A broad `git add -A` swept an in-flight file into a 46-file commit early in the session; later, staging exactly one file by path swept 1424 lines of another agent's gateway work into a commit titled "docs(faq)". The second happened AFTER the first lesson was written into this file. Either use the pathspec form, or read `git diff --cached --stat` immediately before committing and confirm it lists only your files.
- **A git lock here is usually contention, not deadlock.** Something in this environment runs `git status --porcelain` periodically and it takes .git/index.lock (status refreshes the index). It is a live holder, so do NOT remove the lock — retry with a short backoff; it clears in about a second. Verify liveness before ever considering removal, and note that a stat on an absent lock returns 0, which turns an "age" calculation into epoch-seconds that look like data.
- **Prove a mutation applied before believing its result** (diff it). A mutation that gofmt had realigned past reported "ok" and proved nothing.

- **Force-push is forbidden, no exception** (§11.4.113). Integrate by merging onto latest main and push fast-forward only.
- **Every change gets independent review before commit/build** (§11.4.142), iterated to a zero-finding GO (§11.4.134).
- **services/vectorize/__pycache__** and friends are now gitignored — they were one careless git add from being versioned.
- **Rev 24 addition: verify a file edit actually landed before trusting it.** A large inline python3 heredoc updating docs/CONTINUATION.md's metadata table exited 0 with no error but silently changed nothing. The fix was to write the script to a file first, execute it as a separate step, then read the target file back and confirm the change was actually present.

7 · Resume prompt (superseded — see §0/§0b at the top of this file)

This section is rev 23's resume prompt, preserved for history. **Use §0 (short) or §0b (full) at the top of this file instead** — this one's SHAs and push claims are stale as of rev 24 (2026-09-05T18:37:41Z).

Read RESUME.md then specs/002-adaptive-local-model-serving/progress.yml, run ``git fetch --all --prune --tags``, and continue feature 002. Everything is pushed as of rev 23 (meta b752a807, helix_llm 1efda3b5, helix_agent 64cf89 toolkit 328cf27b on a branch) and the system BOOTS – start it with §2b, not ``./helix start``, which is still broken (BOOT-4).

Use subagent-driven development (§11.4.70) by default and fan out on disjoint file scopes. Reproduce every reported finding before fixing it – several turned out to be real and one turned out to be my own regression. Every fix needs a paired mutation, diff-verified as actually applied. Stage by path, and use ``git commit -- <paths>``; staging alone does NOT scope a commit, and other agents share this checkout. Force-push is forbidden (§11.4.113).

NEVER run the test suites while the server is up, and never run several suites at once. Both were measured this session to manufacture false failures – timing, ephemeral-port exhaustion and live-server interference (METHOD-1). this host carries a persistent ~50% background load from OTHER projects (``kfl``, a ``MainThread``, qbittorrent); verify process ownership by cwd before attributing or acting on anything (§11.4.174).